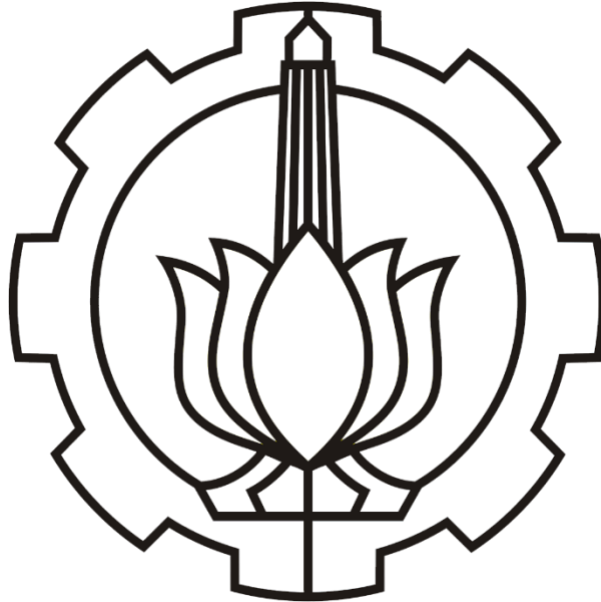


*Dibuat untuk memenuhi tugas Dasar Sistem Cerdas*

## ***Single Layer Perceptron dan Multi Layer Perceptron***



Dosen pembimbing: Fauzan Arrofiqi, S.T, M.T.

Disusun oleh:  
Windy Deftia M  
07311640000015

Teknik Biomedik  
Institut Teknologi Sepuluh Nopember  
2018

## DAFTAR ISI

|                                      |    |
|--------------------------------------|----|
| DASAR TEORI.....                     | 3  |
| Jaringan Syaraf Tiruan.....          | 3  |
| Komponen Jaringan Syaraf Tiruan..... | 3  |
| Arsitektur Jaringan.....             | 4  |
| Fungsi Aktivasi.....                 | 5  |
| Perceptron.....                      | 6  |
| A. Sigle Layer Perceptron .....      | 7  |
| B. Multilayer Perceptron .....       | 9  |
| PERMASALAHAN .....                   | 11 |
| DATA DAN ANALISIS .....              | 12 |
| A. Single layer Perceptron:.....     | 12 |
| B. Multilayer Perceptron:.....       | 13 |
| Percobaan H:.....                    | 14 |
| Percobaan E: .....                   | 15 |
| Percobaan T: .....                   | 17 |
| Percobaan O:.....                    | 19 |
| Perubahan parameter MLP: .....       | 21 |
| KESIMPULAN .....                     | 22 |
| REFERENSI.....                       | 23 |

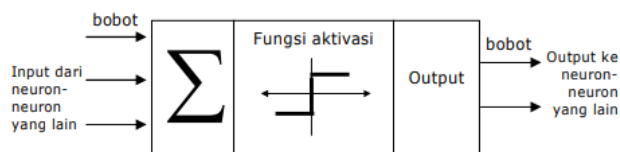
## DASAR TEORI

### Jaringan Syaraf Tiruan

Pemodelan jaringan neural dilakukan dengan pembelajaran dan penyesuaian dari suatu objek dengan tujuan untuk mengenali pola-pola dalam data dan menstimulasikan proses belajar adaptif biologis. Dalam permodelan jaringan neural hal yang perlu diperhatikan adalah banyaknya spesifikasi yang akan diidentifikasi. Model jaringan neural terdiri dari beberapa neuron dan sejumlah masukan untuk menirukan fungsi-fungsi dasar logika. Model-model jaringan neural ini dapat diklasifikasikan menjadi beberapa kriteria diantaranya: menurut metode pembelajaran, menurut *outputnya*, menurut tipe arsitekturnya, dll. Permodelan jaringan neural menurut metode pembelajarannya dibagi menjadi 2 yakni: dengan pengawasan dan tanpa pengawasan. Sedangkan permodelan jaringan neural menurut arsitekturnya dapat dibagi menjadi: feedforward dan berulang. Yang terakhir adalah permodelan jaringan neural menurut *outputnya* dibedakan menjadi biner dan berulang.

### Komponen Jaringan Syaraf Tiruan

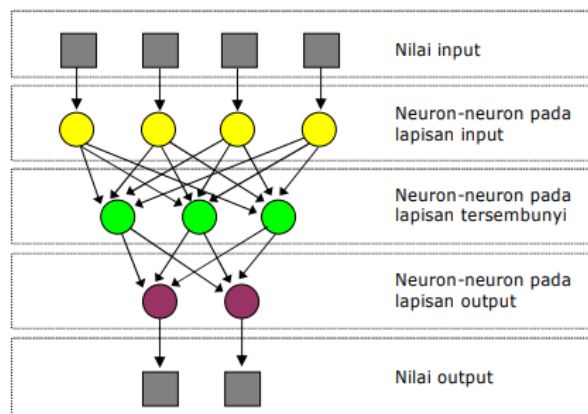
Seperti halnya otak manusia, jaringan syaraf juga terdiri-dari beberapa neuron, dan ada hubungan antara neuron-neuron tersebut. Neuron-neuron tersebut akan mentransformasikan informasi yang diterima melalui sambungan keluarnya menuju ke neuron-neuron yang lain. Pada jaringan syaraf, hubungan ini dikenal dengan nama bobot. Informasi tersebut disimpan pada suatu nilai tertentu pada bobot tersebut.



Gambar 8.2 Struktur neuron jaringan syaraf.

Neuron-neuron buatan tersebut bekerja dengan cara yang sama pula dengan neuron-neuron biologis. Informasi (disebut dengan: input) akan dikirim ke neuron dengan bobot kedatangan tertentu. Input ini akan diproses oleh suatu fungsi perambatan yang akan menjumlahkan nilai-nilai semua bobot yang datang. Hasil penjumlahan ini kemudian akan dibandingkan dengan suatu nilai ambang (threshold) tertentu melalui fungsi aktivasi setiap neuron. Apabila input tersebut melewati suatu nilai ambang tertentu, maka neuron tersebut akan diaktifkan, tapi kalau tidak, maka neuron tersebut tidak akan diaktifkan. Apabila neuron tersebut diaktifkan, maka neuron tersebut akan mengirimkan output melalui bobot-bobot outputnya ke semua neuron yang berhubungan dengannya.

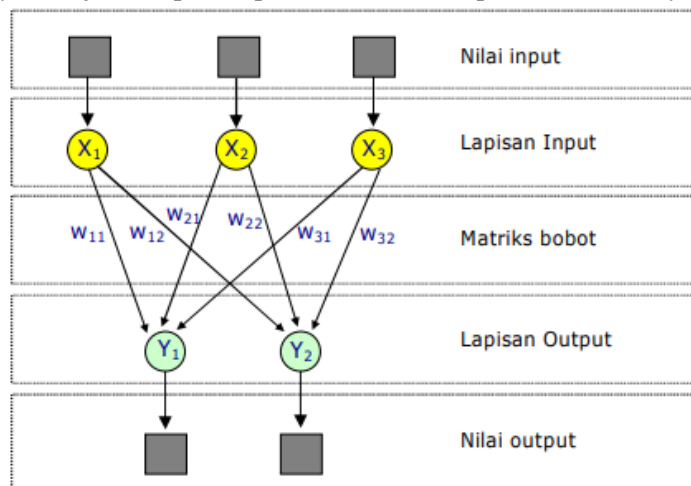
Pada jaringan syaraf, neuron-neuron akan dikumpulkan dalam lapisan-lapisan (layer) yang disebut dengan lapisan neuron (neuron layers). Biasanya neuron-neuron pada satu lapisan akan dihubungkan dengan lapisan-lapisan sebelum dan sesudahnya (kecuali lapisan input dan lapisan output). Informasi yang diberikan pada jaringan syaraf akan dirambatkan lapisan ke lapisan, mulai dari lapisan input sampai ke lapisan output melalui lapisan yang lainnya, yang sering dikenal dengan nama lapisan tersembunyi (hidden layer). Tergantung pada algoritma pembelajarannya, bisa jadi informasi tersebut akan dirambatkan secara mundur pada jaringan.



### Arsitektur Jaringan

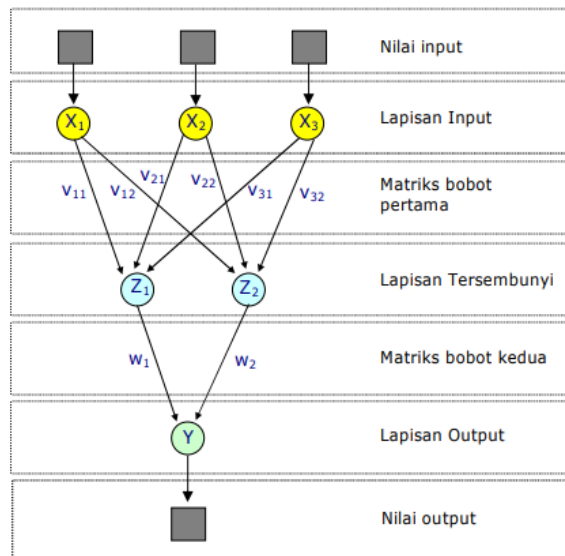
a. Jaringan dengan lapisan tunggal (single layer net)

Jaringan dengan lapisan tunggal hanya memiliki satu lapisan dengan bobot-bobot terhubung. Jaringan ini hanya menerima input kemudian secara langsung akan mengolahnya menjadi output tanpa harus melalui lapisan tersembunyi.



b. Jaringan dengan banyak lapisan (multilayer net)

Jaringan dengan banyak lapisan memiliki 1 atau lebih lapisan yang terletak diantara lapisan input dan lapisan output (memiliki 1 atau lebih lapisan tersembunyi). Umumnya, ada lapisan bobot- bobot yang terletak antara 2 lapisan yang bersebelahan. Jaringan dengan banyak lapisan ini dapat menyelesaikan permasalahan yang lebih sulit daripada lapisan dengan lapisan tunggal, tentu saja dengan pembelajaran yang lebih rumit. Namun demikian, pada banyak kasus, pembelajaran pada jaringan dengan banyak lapisan ini lebih sukses dalam menyelesaikan masalah.

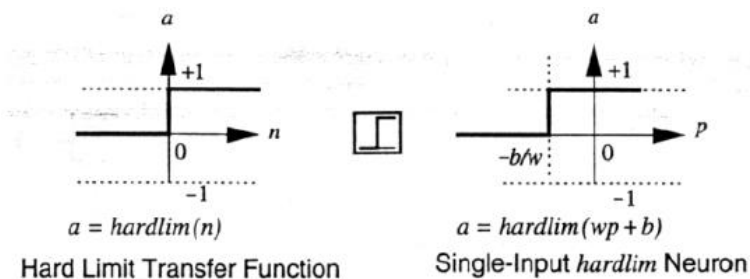


### Fungsi Aktivasi

Fungsi aktivasi digunakan untuk menentukan suatu keluaran dari syaraf tiruan. Terdapat banyak jenis-jenis fungsi aktivasi, diantaranya:

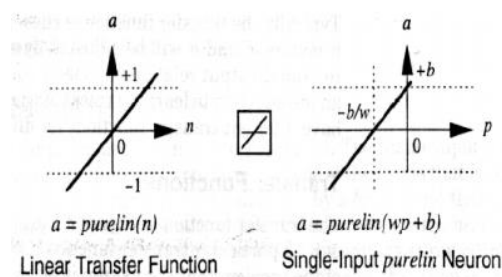
#### 1. Hard limit

Jaringan dengan lapisan tunggal sering menggunakan fungsi undak (step function) untuk mengkonversikan input dari suatu variabel yang bernilai kontinu ke suatu output biner (0 atau 1)



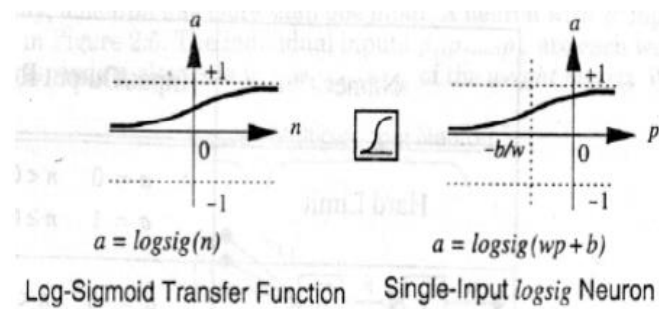
#### 2. Linear

Fungsi linear memiliki nilai output yang sama dengan nilai inputnya



### 3. Log Sigmoid

Fungsi ini digunakan untuk jaringan syaraf yang dilatih dengan menggunakan metode backpropagation. Fungsi sigmoid biner memiliki nilai pada range 0 sampai 1. Oleh karena itu, fungsi ini sering digunakan untuk jaringan syaraf yang membutuhkan nilai output yang terletak pada interval 0 sampai 1. Namun, fungsi ini bisa juga digunakan oleh jaringan syaraf yang nilai outputnya 0 atau 1

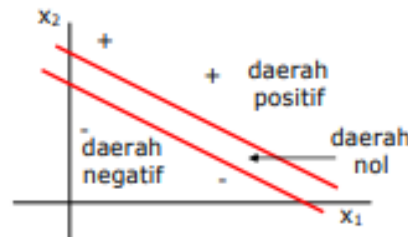


Berikut macam-macam fungsi aktivasi lainnya yang disajikan dalam bentuk tabel:

| Name                        | Input/Output Relation                                                             | Icon | MATLAB Function |
|-----------------------------|-----------------------------------------------------------------------------------|------|-----------------|
| Hard Limit                  | $a = 0 \quad n < 0$<br>$a = 1 \quad n \geq 0$                                     |      | hardlim         |
| Symmetrical Hard Limit      | $a = -1 \quad n < 0$<br>$a = +1 \quad n \geq 0$                                   |      | hardlims        |
| Linear                      | $a = n$                                                                           |      | purelin         |
| Saturating Linear           | $a = 0 \quad n < 0$<br>$a = n \quad 0 \leq n \leq 1$<br>$a = 1 \quad n > 1$       |      | satlin          |
| Symmetric Saturating Linear | $a = -1 \quad n < -1$<br>$a = n \quad -1 \leq n \leq 1$<br>$a = 1 \quad n > 1$    |      | satlins         |
| Log-Sigmoid                 | $a = \frac{1}{1 + e^{-n}}$                                                        |      | logsig          |
| Hyperbolic Tangent Sigmoid  | $a = \frac{e^n - e^{-n}}{e^n + e^{-n}}$                                           |      | tansig          |
| Positive Linear             | $a = 0 \quad n < 0$<br>$a = n \quad 0 \leq n$                                     |      | poslin          |
| Competitive                 | $a = 1 \quad \text{neuron with max } n$<br>$a = 0 \quad \text{all other neurons}$ |      | compet          |

### Perceptron

Perceptron juga termasuk salah satu bentuk jaringan syaraf yang sederhana. Perceptron biasanya digunakan untuk mengklasifikasikan suatu tipe pola tertentu yang sering dikenal dengan pemisahan secara linear. Pada dasarnya, perceptron pada jaringan syaraf dengan satu lapisan memiliki bobot yang bisa diatur dan suatu nilai ambang (threshold). Algoritma yang digunakan oleh aturan perceptron ini akan mengatur parameter-parameter bebasnya melalui proses pembelajaran. Nilai threshold ( $\theta$ ) pada fungsi aktivasi adalah non negatif. Fungsi aktivasi ini dibuat sedemikian rupa sehingga terjadi pembatasan antara daerah positif dan daerah negatif



Garis pemisah antara daerah positif dan daerah nol memiliki pertidaksamaan:  
 $w_1x_1 + w_2x_2 + b > \theta$

Sedangkan garis pemisah antara daerah negatif dengan daerah nol memiliki pertidaksamaan:

$$w_1x_1 + w_2x_2 + b < -\theta$$

Misalkan kita gunakan pasangan vektor input  $s$  dan vektor output sebagai pasangan vektor yang akan dilatih.

Algoritma:

- 1) Inisialisasi semua bobot dan bias: (untuk sederhananya set semua bobot dan bobot bias sama dengan nol). Set learning rate:  $\alpha$  ( $0 < \alpha \leq 1$ ). (untuk sederhananya set sama dengan 1).
- 2) Selama kondisi berhenti bernilai false, lakukan langkah-langkah sebagai berikut:
  - Untuk setiap pasangan pembelajaran  $s$ - $t$ , kerjakan:
    - a. Set input dengan nilai sama dengan vektor input:  $x_i = s_i$ ;
    - b. Hitung respon untuk unit output:
    - c. Perbaiki bobot dan bias jika terjadi error:
 

Jika  $y \neq t$  maka:

$$w_i(\text{baru}) = w_i(\text{lama}) + \alpha * t * x_i$$

$$b(\text{baru}) = b(\text{lama}) + \alpha * t$$

jika tidak, maka:

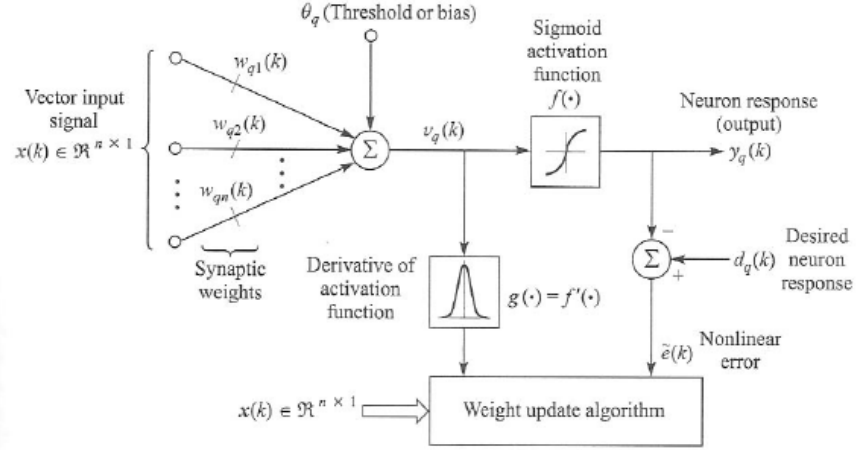
$$w_i(\text{baru}) = w_i(\text{lama})$$

$$b(\text{baru}) = b(\text{lama})$$
  - Tes kondisi berhenti: jika tidak terjadi perubahan bobot pada (i) maka kondisi berhenti TRUE, namun jika masih terjadi perubahan maka kondisi berhenti FALSE.

#### A. Single Layer Perceptron

Single-layer perceptron bisa dibilang merupakan salah satu teknik jaringan saraf tiruan yang paling sederhana. Jaringan ini hanya memiliki lapisan input dan lapisan output. Jaringan ini termasuk supervised learning, artinya metode pembelajarannya

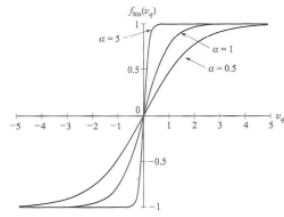
dilakukan melalui contoh-contoh. Jaringan akan ditraining dengan sekumpulan contoh-contoh yang diketahui input dan outputnya. Selama proses belajar tersebut jaringan akan menyesuaikan nilai bobotnya agar menghasilkan output yang diinginkan. Jaringan ini bisa digambarkan sebagai berikut :



Dengan persamaan sebagai berikut:

$$y_q(k) = f[v_q(k)] = f[x^T(k)w_q(k) + \theta_q] = \tanh[\alpha v_q(k)]$$

$$\tilde{e}(k) = d_q(k) - y_q(k)$$



$$y_q(k) = f_{his}(v_q(k)) = \tanh(\alpha v_q(k))$$

$$= \frac{1 - e^{-2\alpha v_q(k)}}{1 + e^{-2\alpha v_q(k)}}$$

$$\begin{aligned} g_{his}(v_q(k)) &= \frac{df_{his}(v_q(k))}{dv_q} \\ &= \alpha[1 + f_{his}(v_q(k))][1 - f_{his}(v_q(k))] \end{aligned}$$

*LMS Learning rule:*

$$\begin{aligned} J(w_q) &= \frac{1}{2} E\{\tilde{e}^2(k)\} \\ &= \frac{1}{2} [d_q(k) - y_q(k)]^2 \\ &= \frac{1}{2} [d_q^2(k) - 2d_q(k)y_q(k) + y_q^2(k)] \end{aligned}$$

*Adaptasi weight:*

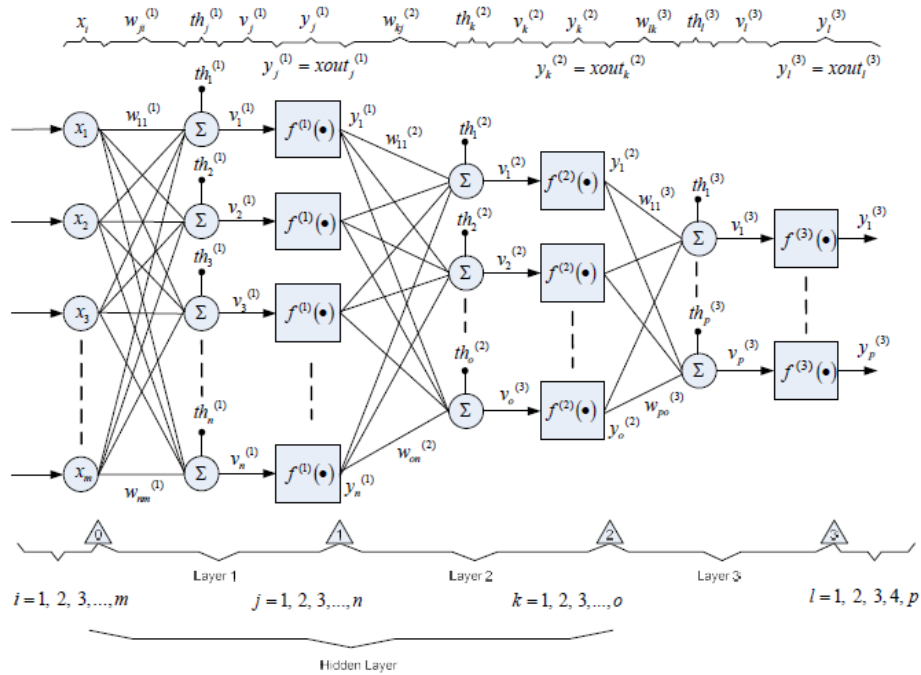
$$\begin{aligned}
w_q(k+1) &= w_q(k) - \mu \nabla J(w_q) \\
\nabla J(w) &\approx \frac{1}{2} \frac{\partial \tilde{e}_q^2(k)}{\partial w} \Big|_{w=w(k)} \\
&= \frac{1}{2} \frac{\partial}{\partial w} [d_q^2(k) - 2d_q(k)y_q(k) + y_q^2(k)] \\
&= \frac{1}{2} \frac{\partial}{\partial w} [d_q^2(k) - 2d_q(k)f[v_q(k)] + f[v_q(k)]f[v_q(k)]] \\
&= -d_q(k)f'[v_q(k)]x(k) + f[v_q(k)]f'[v_q(k)]x(k) \\
&= \{-(d_q(k) - f[v_q(k)])\}f'[v_q(k)]x(k) \\
&= -\tilde{e}_q(k)f'[v_q(k)]x(k) \\
&= -\tilde{e}_q(k)g(v_q(k))x(k)
\end{aligned}$$

$$w_q(k+1) = w_q(k) + \mu \tilde{e}_q(k) \alpha [1 - y_q^2(k)] x(k)$$

$\mu$  = learning rate dan  $\alpha$  = asymptote (slope factor).

#### B. Multilayer Perceptron

Multi-Layer Perceptron adalah jaringan syaraf tiruan feed-forward yang terdiri dari sejumlah neuron yang dihubungkan oleh bobot-bobot penghubung. Neuron-neuron tersebut disusun dalam lapisan-lapisan yang terdiri dari satu lapisan input (inputlayer), satu atau lebih lapisan tersembunyi (hidden layer), dan satu lapisan output (output layer). Multilayer Perceptron diperkenalkan oleh M. Minsky dan S. Papert pada tahun 1969, merupakan pengembangan dari Perceptron dan mempunyai satu atau lebih hidden layers yang terletak antara input dan output layers. Multi-layer-perceptron dapat digunakan untuk menirukan operasi logika yang kompleks seperti XOR.



*Forward multilayer perceptron:*

- Output y dan g pada Layer 1

$$v_j^{(1)} = \left( \sum_{i=1}^m x_i w_{ji}^{(1)} + th_j^{(1)} \right) \rightarrow y_j^{(1)} = f_j^{(1)}[v_j^{(1)}] = \frac{1 - e^{-2\alpha v_j^{(1)}}}{1 + e^{-2\alpha v_j^{(1)}}}$$

$$\rightarrow g_j^{(1)} = \alpha (1 - y_j^{(1)})$$

- Output y dan g pada Layer 2

$$v_k^{(2)} = \left( \sum_{j=1}^n y_j^{(1)} w_{kj}^{(2)} + th_k^{(2)} \right) \rightarrow y_k^{(2)} = f_k^{(2)}[v_k^{(2)}] = \frac{1 - e^{-2\alpha v_k^{(2)}}}{1 + e^{-2\alpha v_k^{(2)}}}$$

$$\rightarrow g_k^{(2)} = \alpha (1 - y_k^{(2)})$$

- Output y dan g pada Layer 3

$$v_l^{(3)} = \left( \sum_{k=1}^o y_k^{(2)} w_{lk}^{(3)} + th_l^{(3)} \right) \rightarrow y_l^{(3)} = f_l^{(3)}[v_l^{(3)}] = \frac{1 - e^{-2\alpha v_l^{(3)}}}{1 + e^{-2\alpha v_l^{(3)}}}$$

$$\rightarrow g_l^{(3)} = \alpha (1 - y_l^{(3)})$$

*Backpropagation multilayer perceptron:*

- Mencari delta error tiap layer

$$E = \frac{1}{2} (d - y^{(3)})^2$$

$$= \frac{1}{2} \sum_{l=1}^p (d_l - y_l^{(3)})^2$$

- Mencari delta error layer 3

$$\delta_l^{(3)} = \frac{\partial E}{\partial v_l^{(3)}} = (d_l - y_l^{(3)}) g_l^{(3)}$$

- Mencari delta error layer 2

$$\delta_k^{(2)} = \frac{\partial E}{\partial v_k^{(2)}} = \left( \sum_{l=1}^p \delta_l^{(3)} w_{lk}^{(3)} \right) g_k^{(2)}$$

- Mencari delta error layer 1

$$\delta_j^{(1)} = \frac{\partial E}{\partial v_j^{(1)}} = \left( \sum_{k=1}^o \delta_k^{(2)} w_{kj}^{(2)} \right) g_j^{(1)}$$

## **PERMASALAHAN**

Metode perceptron merupakan salah satu metode untuk meniru jaringan syaraf manusia. Metode ini dapat meniru logika-logika, seperti AND. Bagaimana cara menyajikan metode single layer perceptron dan multilayer perceptron kedalam sebuah program. Kemudian menyajikan pengenalan terhadap karakter E, T, H, O dan point yang mana Setiap karakter memiliki 144 piksel dengan dimensi 12x12. Kemudian Bagian yang akan ditampilkan agar membentuk suatu karakter disimbolkan dengan logika 1, sedangkan bagian yang tidak ditampilkan disimbolkan dengan logika 0. Setiap karakter didefinisikan sebagai array satu dimensi yang berisi 144 bit data.

## DATA DAN ANALISIS

### A. Single layer Perceptron:

Tampilan Interface awal dari single layer perceptron adalah sebagai berikut:

Tombol *new data* untuk menyimpan *weight* dan *threshold* awal, sehingga jika diganti dan dilakukan analisis terhadap perubahan parameter hasilnya akan stabil. Kemudian tombol *load data* berguna untuk membuka data *weight* serta *threshold* yang sudah disimpan sebelumnya. Untuk tombol *save file* dan *load file* berguna untuk menyimpan dan membuka file yang berisi parameter-parameter serta *weight* dan *threshold* awal.

### Percobaan:

Th= 0,0116307809549063

W1=0,00483804679901884

W2=0,00299563510850273

| Miu | Tau  | Alpha | Epsilon | MSE                  | iterasi |
|-----|------|-------|---------|----------------------|---------|
| 10  | 1000 | 2     | 0,0001  | 0,000415676780111381 | 1132    |
| 50  | 1000 | 2     | 0,0001  | 0,499999999998889    | 1180    |
| 100 | 1000 | 2     | 0,0001  | 0,5                  | 1188    |
| 10  | 100  | 2     | 0,0001  | 0,00120729254714044  | 1080    |
| 10  | 500  | 2     | 0,0001  | 0,000736894725431435 | 1092    |
| 10  | 1000 | 2     | 0,0001  | 0,000422686843706865 | 1096    |
| 10  | 2000 | 2     | 0,0001  | 0,000231897452525    | 1072    |

|    |      |   |        |                      |      |
|----|------|---|--------|----------------------|------|
| 10 | 1000 | 1 | 0,0001 | 0,00237374692022521  | 1092 |
| 10 | 1000 | 2 | 0,0001 | 0,000429231355807543 | 1064 |
| 10 | 1000 | 5 | 0,0001 | 0,499999999999987    | 1076 |
| 10 | 1000 | 2 | 0,0001 | 0,000429231355807543 | 1064 |
| 10 | 1000 | 2 | 0,001  | 0,000989719107198424 | 276  |
| 10 | 1000 | 2 | 0,01   | 0,00749430038426632  | 144  |

Dari data yang didapatkan maka dapat dilihat pengaruh  $\mu$  terhadap MSE, semakin besar  $\mu$  maka semakin besar pula MSE. Hal ini berarti nilai saat *testing* akan semakin menjauhi logika AND yang sebenarnya. Selain itu pengaruh parameter tau, dari data diatas kita ketahui bahwa semakin besar tau maka MSE akan semakin kecil, hal ini berarti nilai logika AND saat *testing* akan semakin mendekati nilai sebenarnya. Kemudian untuk parameter alpha menunjukkan ketidak stabilan dari MSE. Dari rentang alpha 1 sampai dengan 5 maka didapatnya alpha paling baik ada pada angka 2. Ketika dicoba saat  $\alpha = 5$  maka MSE akan meningkat drastis, yang berarti nilai dari logika AND saat *testing* akan semakin menjauhi nilai aslinya. Yang terakhir adalah Epsilon, epsilon sangat berpengaruh pada iterasi, semakin besar epsilon maka pembelajaran yang dilakukan semakin cepat. Walaupun pembelajaran semakin cepat namun nilai MSE juga semakin bertambah, sehingga dapat diartikan semakin besar epsilon ketelitian terhadap hasil logika sebenarnya semakin berkurang.

## B. Multilayer Perceptron:

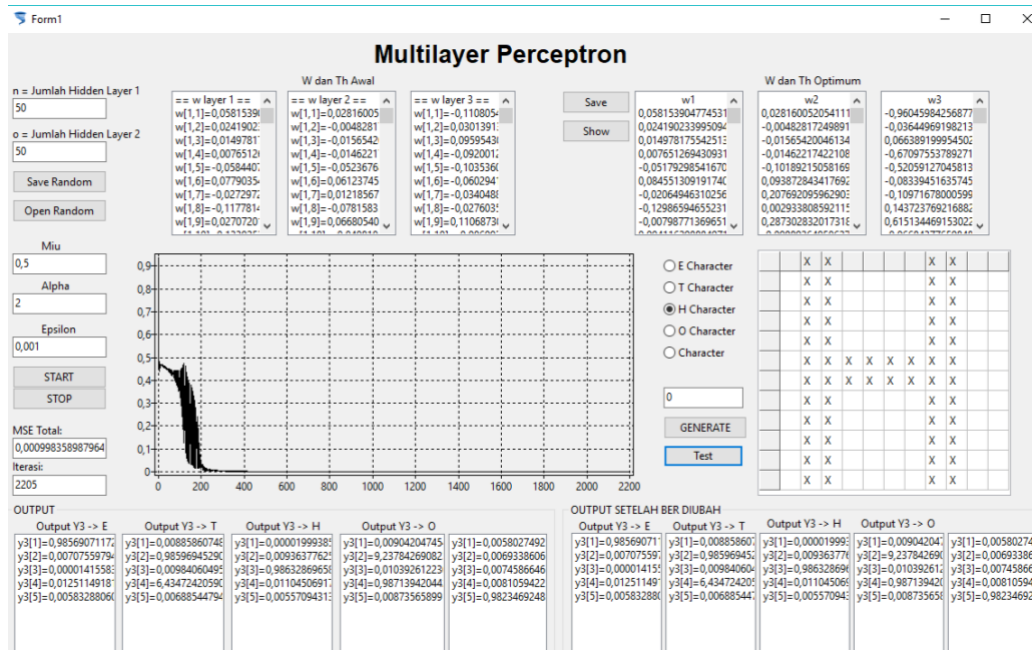
Tampilan awal interface dari program multilayer perceptron adalah sebagai berikut:

Tombol save dan open random berguna untuk menyimpan inisialisasi weight dan threshold awal. Kemudian jumlah hidden layer dapat ditentukan pada kolom “n” dan “o”. Sebelum menjalankan program pilih huruf apa yang ingin diidentifikasi atau dikenali.

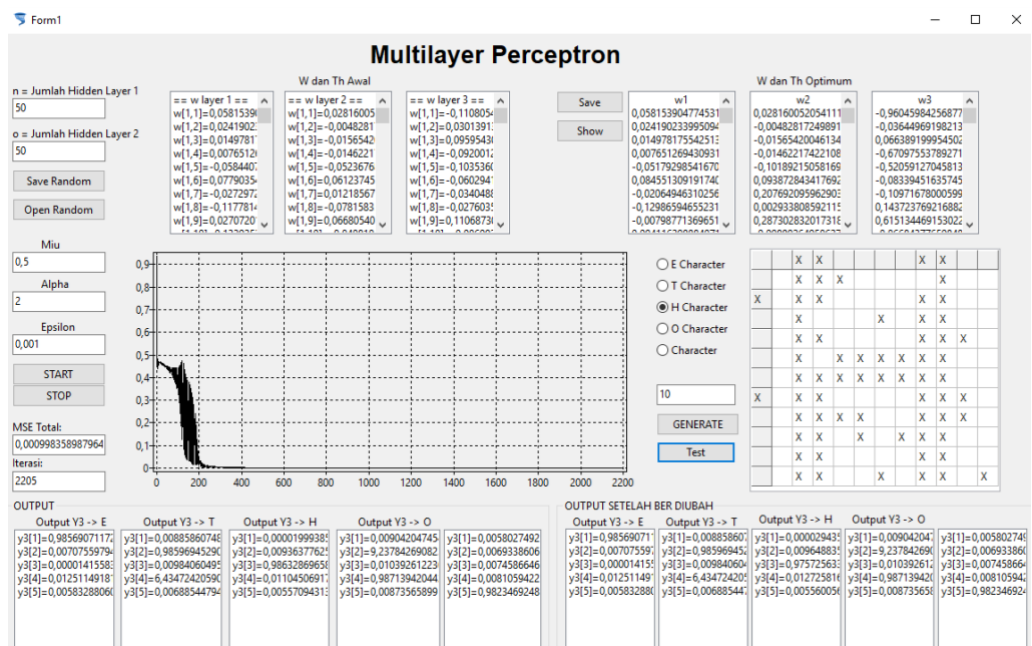
## Percobaan H:

Weight dan threshold awal diambil dari file yang sudah disimpan sebelumnya.

Mulanya dilakukan testing saat bit error H = 0 kemudian hasil yang didapat adalah sama dengan output awal.



Kemudian dilakukan penggantian bit error untuk huruf H sebesar 10

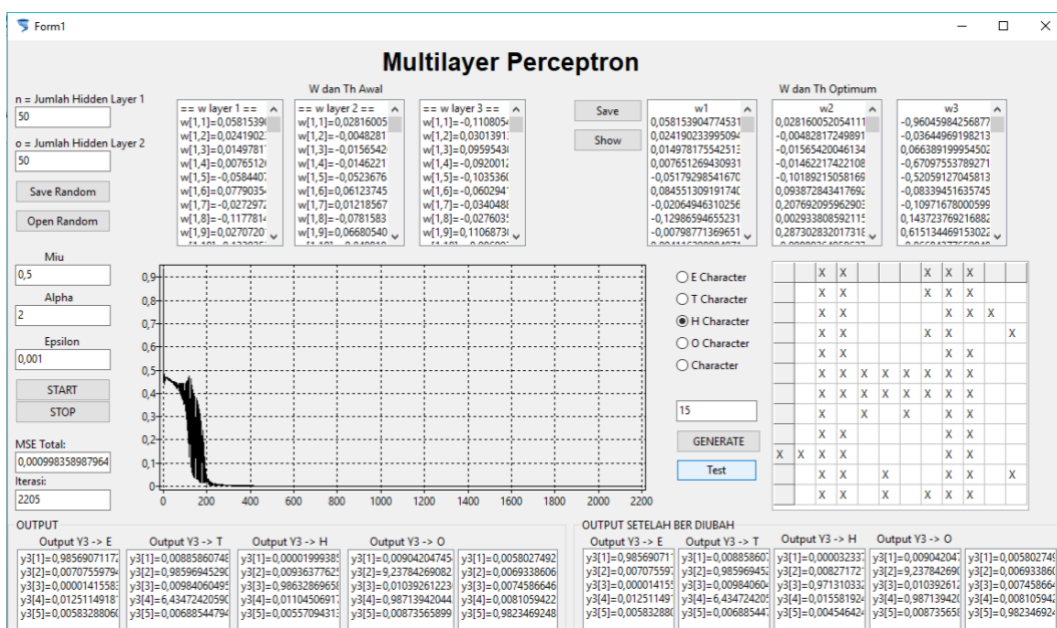


| OUTPUT             |                    |                    |                    |                    |
|--------------------|--------------------|--------------------|--------------------|--------------------|
| Output Y3 -> E     | Output Y3 -> T     | Output Y3 -> H     | Output Y3 -> O     |                    |
| y3[1]=0,9856907117 | y3[1]=0,0088586074 | y3[1]=0,0000199938 | y3[1]=0,0090420474 | y3[1]=0,0058027492 |
| y3[2]=0,0070755979 | y3[2]=0,9859694529 | y3[2]=0,0093637762 | y3[2]=9,2378426908 | y3[2]=0,0069338606 |
| y3[3]=0,0000141558 | y3[3]=0,0098406049 | y3[3]=0,9863286965 | y3[3]=0,0103926122 | y3[3]=0,0074586646 |
| y3[4]=0,0125114918 | y3[4]=6,4347242059 | y3[4]=0,0110450691 | y3[4]=0,9871394204 | y3[4]=0,0081059422 |
| y3[5]=0,0058328806 | y3[5]=0,0068854479 | y3[5]=0,0055709431 | y3[5]=0,0087356589 | y3[5]=0,9823469248 |

| OUTPUT SETELAH BER DIUBAH |                  |                  |                  |                  |
|---------------------------|------------------|------------------|------------------|------------------|
| Output Y3 -> E            | Output Y3 -> T   | Output Y3 -> H   | Output Y3 -> O   |                  |
| y3[1]=0,98569071          | y3[1]=0,00885860 | y3[1]=0,00002943 | y3[1]=0,00904204 | y3[1]=0,00580274 |
| y3[2]=0,00707559          | y3[2]=0,98596945 | y3[2]=0,00964883 | y3[2]=9,23784269 | y3[2]=0,00693386 |
| y3[3]=0,00001415          | y3[3]=0,00984060 | y3[3]=0,97572563 | y3[3]=0,01039261 | y3[3]=0,00745866 |
| y3[4]=0,01251149          | y3[4]=6,43472420 | y3[4]=0,01272581 | y3[4]=0,98713942 | y3[4]=0,00810594 |
| y3[5]=0,00583288          | y3[5]=0,00688544 | y3[5]=0,00556005 | y3[5]=0,00873565 | y3[5]=0,98234692 |

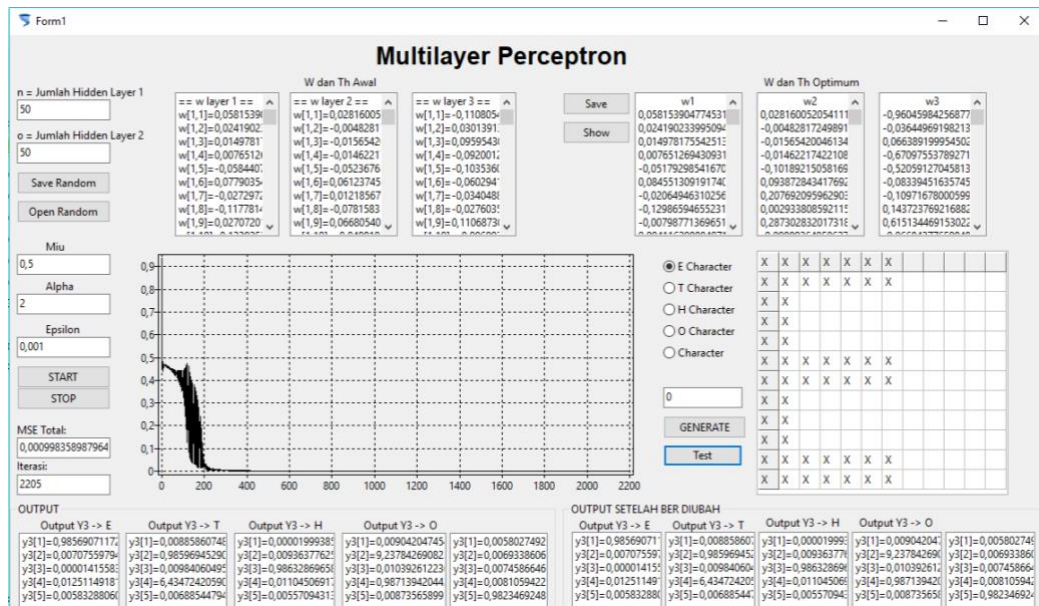
Jika dilihat perbedaan yang ada pada kolom “Output Y3 -> H” tidak terlalu signifikan sehingga program masih dapat mengenali huruf H. Namun setelah dilakukan *trial and error* pada *bit error*, didapatkan **bit error maksimum** untuk program mengenali huruf H adalah sebesar **15**. Dengan tampilan sebagai berikut:



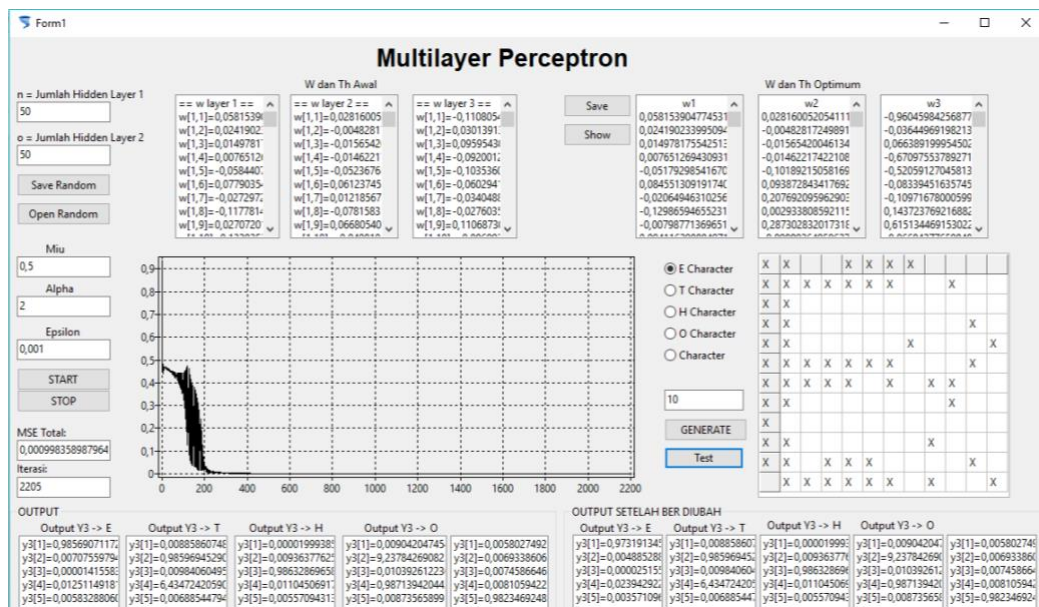
### Percobaan E:

Weight dan threshold awal diambil dari file yang sudah disimpan sebelumnya.

Mulanya dilakukan pengecekan terhadap bit error E ketika BER= 0, hasil yang didapat adalah sama dengan output program awal, seperti yang terlihat pada gambar dibawah ini:



Kemudian dilakukan perubahan BER sebesar 10 dan didapatkan hasil sebagai berikut:



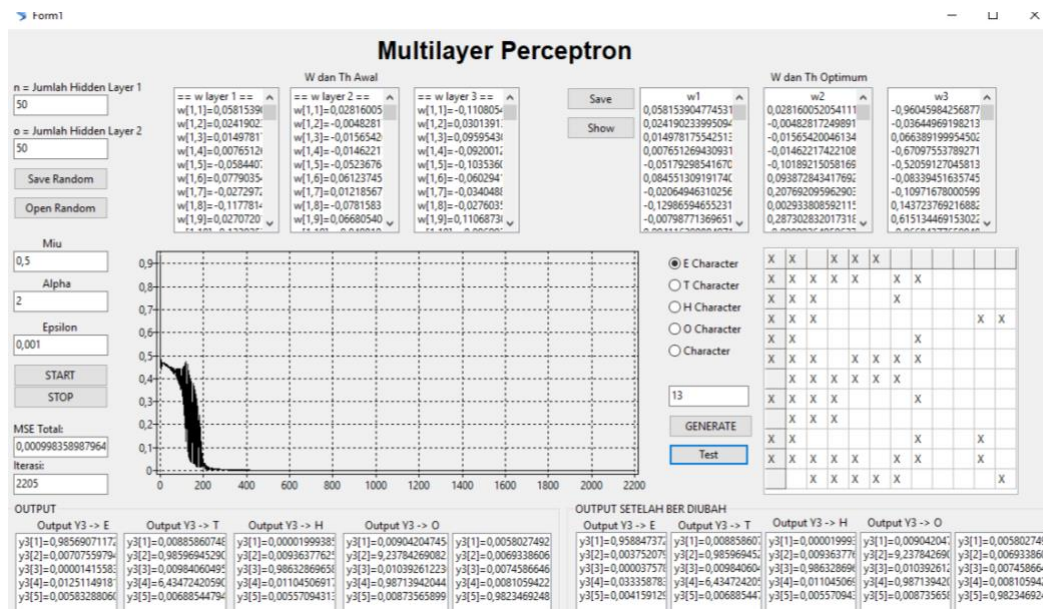
#### OUTPUT

| Output Y3 -> E     | Output Y3 -> T      | Output Y3 -> H     | Output Y3 -> O      |
|--------------------|---------------------|--------------------|---------------------|
| y3[1]=0.9856907117 | y3[1]=0.00885860745 | y3[1]=0.0000199938 | y3[1]=0.00904204745 |
| y3[2]=0.0070755979 | y3[2]=0.9859694529  | y3[2]=0.0093637762 | y3[2]=0.0069338606  |
| y3[3]=0.0000141558 | y3[3]=0.0098406049  | y3[3]=0.9863286965 | y3[3]=0.01039261223 |
| y3[4]=0.0125114918 | y3[4]=6.4347242059  | y3[4]=0.0110450691 | y3[4]=0.0081059422  |
| y3[5]=0.0058328806 | y3[5]=0.00688544794 | y3[5]=0.0055709431 | y3[5]=0.00873565899 |

#### OUTPUT SETELAH BER DIUBAH

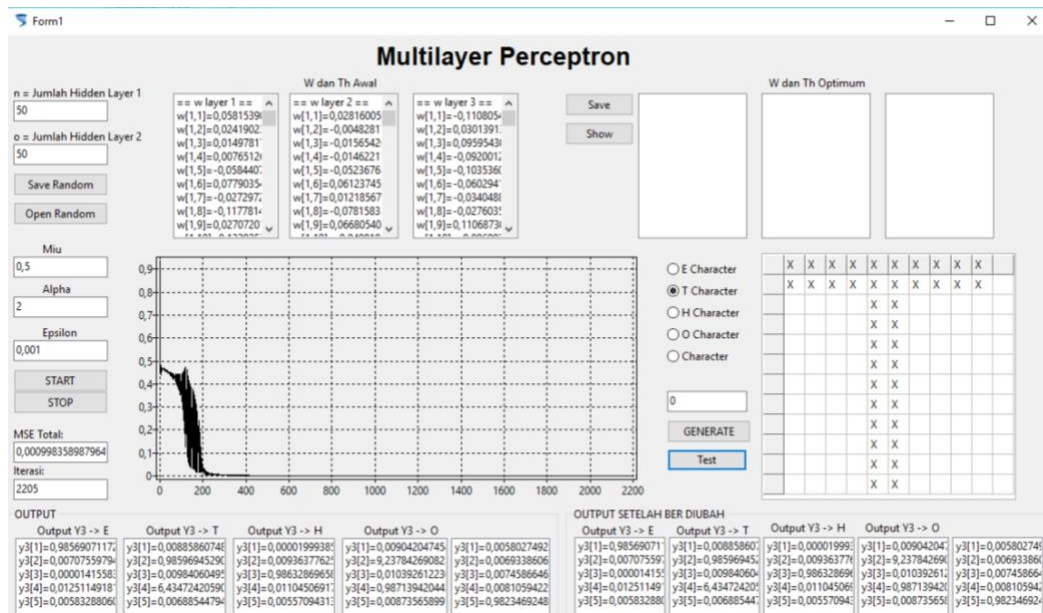
| Output Y3 -> E   | Output Y3 -> T      | Output Y3 -> H     | Output Y3 -> O      |
|------------------|---------------------|--------------------|---------------------|
| y3[1]=0.97319134 | y3[1]=0.00885860745 | y3[1]=0.0000199938 | y3[1]=0.00904204745 |
| y3[2]=0.00488528 | y3[2]=0.9859694529  | y3[2]=0.0093637762 | y3[2]=0.0069338606  |
| y3[3]=0.00002515 | y3[3]=0.0098406049  | y3[3]=0.9863286965 | y3[3]=0.01039261223 |
| y3[4]=0.02394292 | y3[4]=6.4347242059  | y3[4]=0.0110450691 | y3[4]=0.0081059422  |
| y3[5]=0.00357109 | y3[5]=0.00688544794 | y3[5]=0.0055709431 | y3[5]=0.00873565899 |

Perubahan terhadap huruf E masih dapat diidentifikasi karena error output terbilang kecil. Selanjutnya seperti sebelumnya, dilakukan *trial and error* terhadap *bit error* dan didapatkan **bit error maksimum** program untuk mengidentifikasi huruf E sebesar **13**, seperti pada gambar dibawah ini:

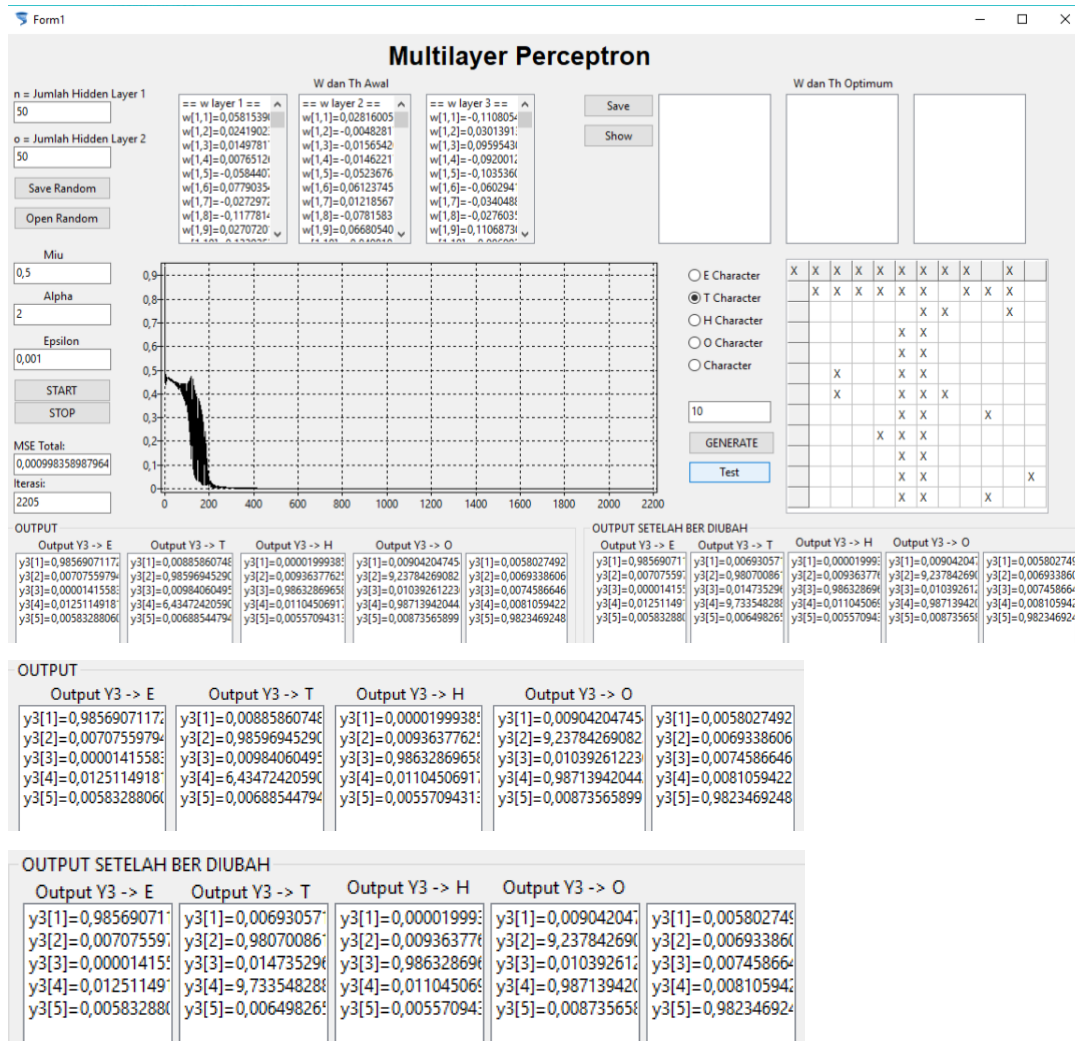


### Percobaan T:

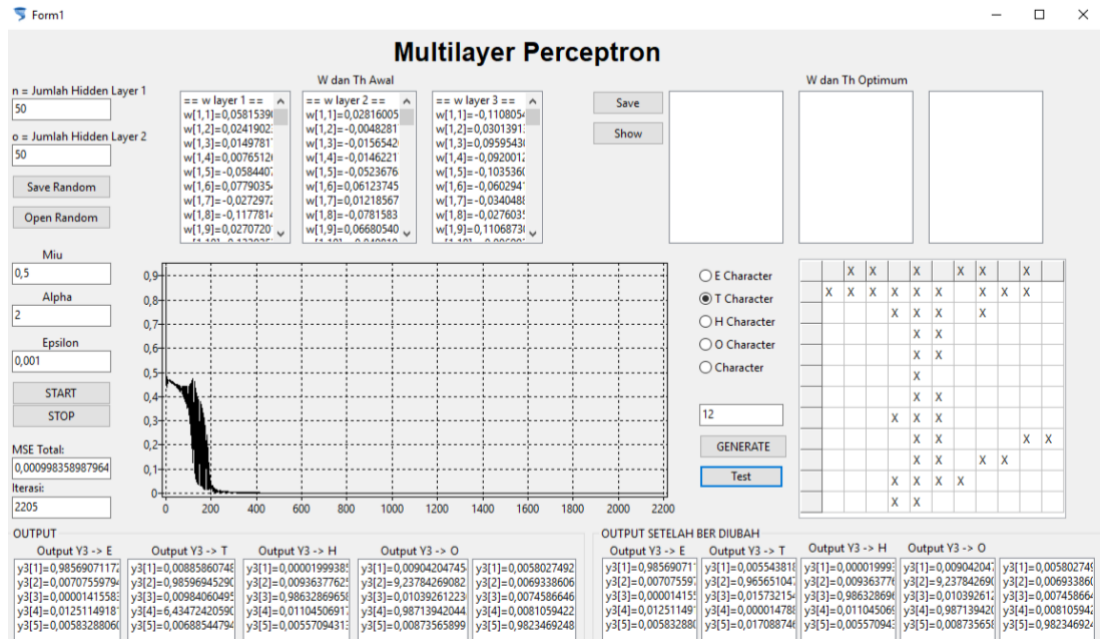
Mulanya dilakukan pengecekan terhadap bit error T ketika BER= 0, hasil yang didapat adalah sama dengan output program awal, seperti yang terlihat pada gambar dibawah ini:



Kemudian dilakukan perubahan terhadap BER menjadi 10:

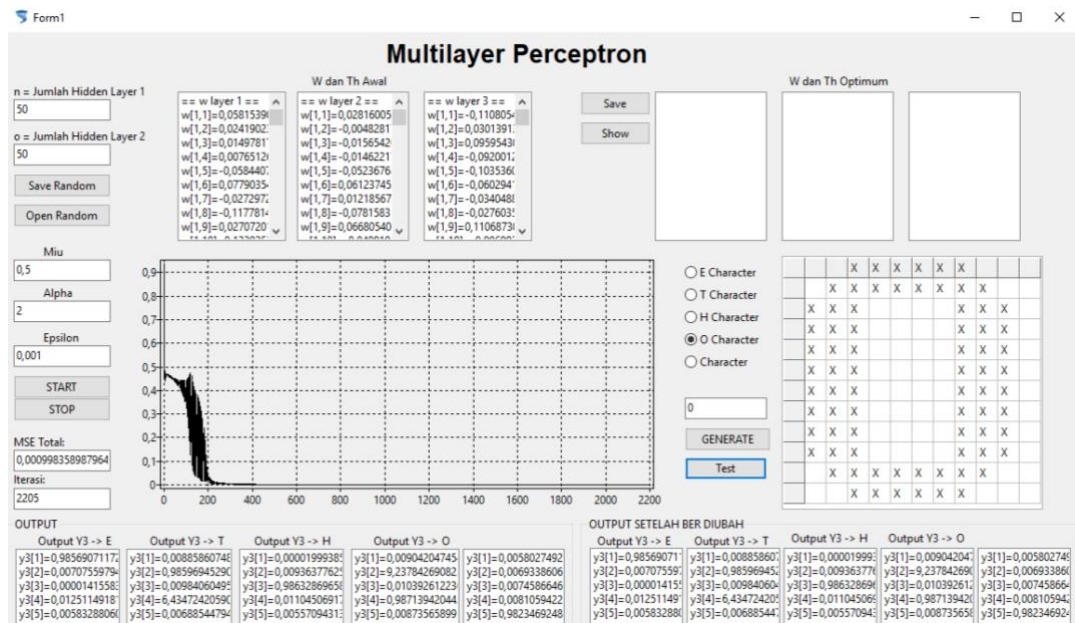


Perubahan terhadap huruf T masih dapat diidentifikasi karena error tidak terlalu signifikan. Kemudian, dilakukan *trial and error* terhadap *bit error* dan didapatkan **bit error maksimum** program untuk mengidentifikasi huruf T sebesar **12**, seperti pada gambar dibawah ini:

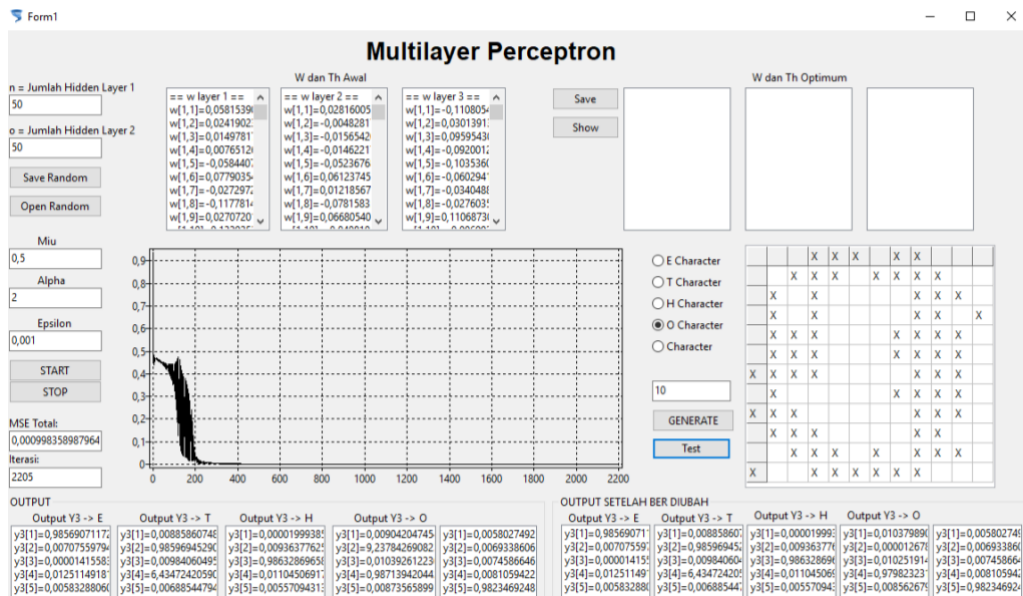


### Percobaan O:

Mulanya dilakukan pengecekan terhadap bit error T ketika BER= 0, hasil yang didapat adalah sama dengan output program awal, seperti yang terlihat pada gambar dibawah ini:



Kemudian dilakukan perubahan BER sebesar 10



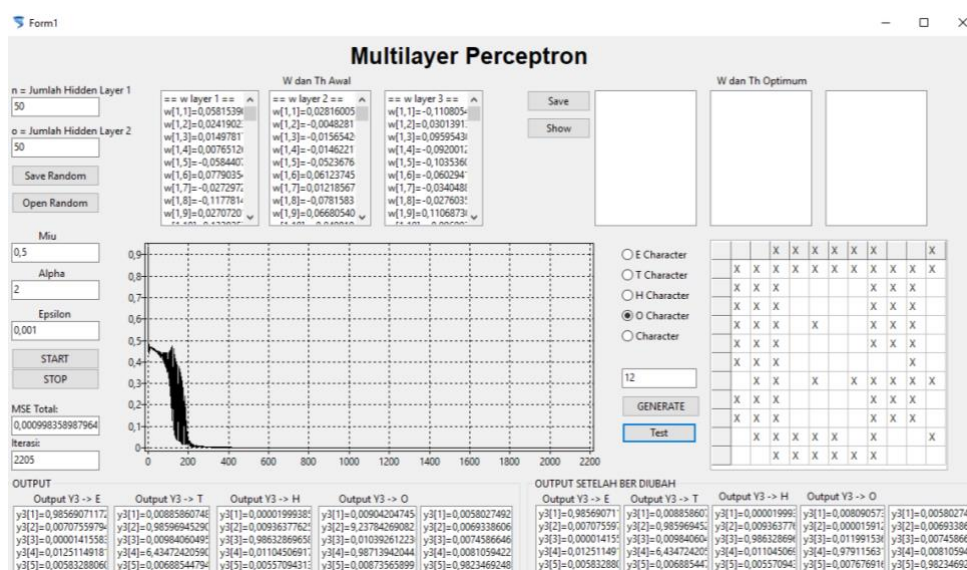
OUTPUT

| Output Y3 -> E     | Output Y3 -> T     | Output Y3 -> H     | Output Y3 -> O     |
|--------------------|--------------------|--------------------|--------------------|
| y3[1]=0,9856907117 | y3[1]=0,0088586074 | y3[1]=0,0000199938 | y3[1]=0,0090420474 |
| y3[2]=0,0070755979 | y3[2]=0,9859694529 | y3[2]=0,0093637762 | y3[2]=9,2378426908 |
| y3[3]=0,0000141558 | y3[3]=0,0098406049 | y3[3]=0,9863286958 | y3[3]=0,0103926123 |
| y3[4]=0,0125114918 | y3[4]=6,4347242059 | y3[4]=0,0110450691 | y3[4]=0,9871394204 |
| y3[5]=0,0058328806 | y3[5]=0,0068854479 | y3[5]=0,0055709431 | y3[5]=0,0087356589 |

OUTPUT SETELAH BER DIUBAH

| Output Y3 -> E   | Output Y3 -> T   | Output Y3 -> H   | Output Y3 -> O   |
|------------------|------------------|------------------|------------------|
| y3[1]=0,98569071 | y3[1]=0,00885860 | y3[1]=0,00001999 | y3[1]=0,01037989 |
| y3[2]=0,00707559 | y3[2]=0,98596945 | y3[2]=0,00936377 | y3[2]=0,00001267 |
| y3[3]=0,00001415 | y3[3]=0,00984060 | y3[3]=0,98632869 | y3[3]=0,01025191 |
| y3[4]=0,01251149 | y3[4]=6,43472420 | y3[4]=0,01104506 | y3[4]=0,97982323 |
| y3[5]=0,00583288 | y3[5]=0,00688544 | y3[5]=0,00557094 | y3[5]=0,00856267 |

Perubahan terhadap huruf O masih dapat diidentifikasi karena error output terbilang kecil. Selanjutnya seperti sebelumnya, dilakukan *trial and error* terhadap *bit error* dan didapatkan **bit error maksimum** program untuk mengidentifikasi huruf O sebesar **12**, seperti pada gambar dibawah ini:



**Perubahan parameter MLP:**

| Hidden Layer | Miu | Alpha | Epsilon | MSE                  | Iterasi |
|--------------|-----|-------|---------|----------------------|---------|
| 50           | 0,5 | 2     | 0,0001  | 0,000102780911950524 | >15000  |
| 50           | 0,5 | 2     | 0,001   | 0,000998358987964393 | 2205    |
| 50           | 0,5 | 2     | 0,01    | 0,00996236186011909  | 400     |
| 50           | 0,5 | 2     | 0,1     | 0,0838830539589282   | 210     |
| 50           | 0,5 | 1     | 0,001   | 0,000999592386955401 | 9015    |
| 50           | 0,5 | 2     | 0,001   | 0,000998358987964393 | 2205    |
| 50           | 0,5 | 5     | 0,001   | 3,99999972498507     | 5403    |
| 50           | 0,1 | 2     | 0,001   | 0,000999842214328555 | 10425   |
| 50           | 0,5 | 2     | 0,001   | 0,000998358987964393 | 2205    |
| 50           | 1   | 2     | 0,001   | 0,000998831890263496 | 1385    |

Dari data yang didapatkan diatas maka dapat diketahui bahwa perubahan parameter epsilon sangat berpengaruh terhadap iterasi atau ecepatan pembelajaran dari ANN Multilayer perceptron. Namun, perubahan epsilon yang semakin besar juga berpengaruh pada besaran MSE, semakin besar epsilon maka error pun juga semakin besar. Setelah itu, untuk parameter alpha sangat berpengaruh pada error. Terlihat ketika  $\alpha=5$  error meningkat drastis menjadi  $\cong 4$ . Kemudian untuk perubahan miu juga berpengaruh pada iterasi atau lamanya pembelajaran dari ANN, namun dari data diatas juga dapat dilihat bahwa pada perubahan miu terdapat ketidak stabilan dari MSE.

## **KESIMPULAN**

Metode perceptron dapat menirukan gerbang logika, multilayer perceptron khususnya, dapat menirukan hingga gerbang logika yang kompleks seperti XOR. Parameter-parameter yang ada pada single layer perceptron maupun multilayer perceptron memiliki kegunaan masing-masing sehingga mempengaruhi kecepatan pembelajaran dari ANN maupun error dari output ANN

## REFERENSI

1. [http://share.its.ac.id/pluginfile.php/14163/mod\\_resource/content/1/jst1.pdf](http://share.its.ac.id/pluginfile.php/14163/mod_resource/content/1/jst1.pdf)  
Diakses pada tanggal 06 Juni 2018 pukul 20.00
2. <https://igawidagda.files.wordpress.com/2012/02/diktat-jst.pdf> Diakses pada  
tanggal 06 Juni 2018 pukul 20.00
3. [http://dinus.ac.id/repository/docs/ajar/Pengertian\\_Multi\\_Layer\\_Neural\\_Network.pdf](http://dinus.ac.id/repository/docs/ajar/Pengertian_Multi_Layer_Neural_Network.pdf) Diakses pada tanggal 06 Juni 2018 pukul 20.00
- 4.